a simple *append* method with which to add a new element. There are many more operations possible with a list, which we could look into in later columns.

When set up for the above purpose, the code looks like what follows:

```
incorrectlines = []
for section in filetovalidate.sections():
#check each key is present in corresponding golden section
for key in filetovalidate.options(section):
if not goldenconfig.has_option(section,key):
incorrectlines.append(key +'='+filetovalidate.
get(section,key))
```

When we execute the same, we get the following output in IDLE GUI:

```
>>> for section in filetovalidate.sections():
        for key in filetovalidate.options(section):
            if not goldenconfig.has_option(section,key):
                incorrectlines.append(key
+'='+filetovalidate.get(section,key))


>>> incorrectlines
['custome=CustomerX', 'brandingtitl=CustomerX Title',
'customcontact=CustomerX contact', 'tomercontact=CustomerY
contact']
```

We could choose to print these into the *stdout* for now. Note that we can simply return a list of *incorrectlines* as part of our function execution.

```
if len(incorrectlines) > 0 :
        print 'The following lines are incorrect'
        for k in incorrectlines: print k
else: print 'The config file is fine'
return incorrectlines
```

*len(list)* returns the number of elements in the list. If this is non-zero, we iterate through the list using a *for* construct and print each value as we iterate. If there are no incorrect lines, we print that the config file is fine.

Executing the same in IDLE GUI returns the following output:

```
>>> len(incorrectlines)
4
>>> if len(incorrectlines) > 0 :
          print 'The following lines are incorrect'
          for k in incorrectlines: print k
else: print 'The config file is fine'

The following lines are incorrect
```

***Note:*** In an ideal function, we only return the lines that are erroneous. We should not be making any prints into *stdout* as the function caller will not be expecting any such behaviour.

```
custome=CustomerX
brandingtitl=CustomerX Title
customcontact=CustomerX contact
tomercontact=CustomerY contact
```

## Testing the function

We could add some self-test documentation in the source file to illustrate the usage.

We would expect *ValidateConfigFile('c:\example.cfg','c:\example.cfg')* to return an empty list.

```
print "invoking ValidateConfigFile('validfile','validfile')"
lines = ValidateConfigFile('c:\example.cfg','c:\example.cfg')
if len(lines)==0: print 'self test ok'

lines = ValidateConfigFile('c:\example.cfg','c:\error.cfg')
if len(lines)>0: print 'errors present'
```

## Documentation strings

We can add documentation strings in the following way:

```
"""
Validates a given Config File in filetobevalidated against a
correct config file pointed to by goldenfilepath
returns a list of erroneous lines as a list[strings]
if config file is fine, it should return an empty list
len(ValidateFile('c:\example.cfg', 'c:\example.cfg' ))== 0
"""
```

## The complete file listing

This is as follows:

```
#ValidateConfigFile.py
import ConfigParser
def ValidateConfigFile(goldenfilepath = 'none',
filetobevalidated = 'none' ) :
"""
Validates a given Config File in filetobevalidated against a
correct config file pointed to by goldenfilepath
returns a list of erroneous lines as a list[strings]
if config file is fine, it should return an empty list
len(ValidateFile('c:\example.cfg', 'c:\example.cfg' ))== 0
"""

#learn golden file
goldenconfig = ConfigParser.ConfigParser()
goldenconfig.read(goldenfilepath)

#learn file to be validated
```